

ChatGPT



how to create knowledge base ai assistant?

Creating a knowledge-base AI assistant means combining a language model with your own documents, policies, manuals, or company data so it can answer questions accurately.

Basic Architecture

A typical knowledge-base assistant has these components:

1. Knowledge source

- PDFs
- Word documents
- Wikis
- Databases
- Websites
- FAQs

2. Document processing

- Extract text
- Split into chunks
- Store metadata

3. Vector database

- Converts text into embeddings
- Enables semantic search
- Examples:
 - Pinecone
 - Weaviate
 - Qdrant
 - Chroma

4. LLM (Language Model)

- Receives user question
- Retrieves relevant knowledge
- Generates answer

5. Chat interface

- Web app
- Slack bot
- Teams bot
- Internal portal

This approach is commonly called Retrieval-Augmented Generation (RAG), a form of Retrieval-Augmented Generation.

Easiest No-Code Method

If you don't want to write code:

Option 1: OpenAI GPTs

Use:

[OpenAI GPTs Builder](#)

Steps:

1. Create a custom GPT.
2. Upload PDFs, manuals, FAQs, or documentation.
3. Add instructions describing the assistant's role.
4. Test questions.
5. Share privately or publicly.

Suitable for:

- HR assistants
 - Product documentation bots
 - Internal company knowledge assistants
-

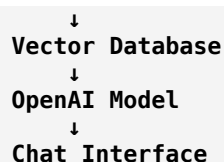
Low-Code Method

Tools that can build RAG assistants quickly:

- [Flowise](#)
- [Langflow](#)
- [Dify](#)

Typical workflow:

Documents
↓
Embeddings



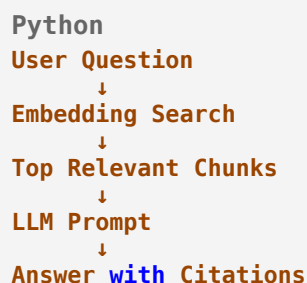
You can usually get a prototype running in a few hours.

Build It Yourself (Developer Approach)

Popular stack:

- LLM: [OpenAI API](#)
- Framework: [LangChain](#) or [LlamaIndex](#)
- Vector DB: Qdrant, Pinecone, Weaviate
- Frontend: React, Next.js, or a chatbot widget

Example flow:



Best Practices

1. Keep documents updated

AI answers are only as good as the source material.

2. Return sources

Show:

- Document name
- Page number
- URL

This increases trust.

3. Use metadata

Store:

- Department
- Date
- Product
- Author

Then filter searches before retrieval.

4. Chunk correctly

Typical chunk size:

- 300–800 tokens

Too large = poor retrieval.

Too small = lost context.

5. Add permissions

For company assistants:

- Employees see only authorized documents.
 - Integrate with SSO.
-

Example Use Cases

- Customer support assistant
 - Employee handbook assistant
 - IT help desk bot
 - Product documentation chatbot
 - Legal document search assistant
 - Research assistant
 - Healthcare knowledge assistant
-

Example Project

Suppose you have:

- 500 PDFs
- Product manuals
- SOPs
- FAQ documents

Workflow:

1. Upload documents.
2. Extract text.
3. Generate embeddings.
4. Store in Qdrant.
5. User asks:

"How do I reset Product X?"
6. System retrieves the most relevant manual sections.
7. LLM answers and cites the manual pages.

This usually produces much more reliable answers than relying on the language model's training data alone.

If you tell me:

- your use case (company knowledge, customer support, legal, healthcare, etc.),
- expected number of documents,
- whether you want no-code or coding,

I can provide a specific architecture and implementation plan.



I have the following stack/tools

Vs Code
 Visual Studio
 OLLAMA
 React TypeScript
 MSSQL
 langflow
 Docker Desktop
 qDRANT(Docker)
 CLAUDE DESKTOP
 Show moreShow less

You already have everything needed to build a production-quality local knowledge-base assistant.

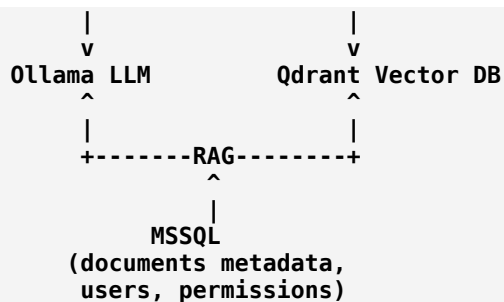
Recommended Architecture

React + TypeScript Frontend



Backend API (.NET or Node.js)





Your Tools' Roles

Tool	Purpose
VS Code	React frontend, scripts, LangFlow flows
Visual Studio	ASP.NET Core backend (recommended)
Ollama	Local LLM inference
React + TypeScript	Chat UI
MSSQL	Document metadata, users, audit logs
LangFlow	RAG workflow design and testing
Docker Desktop	Run infrastructure
Qdrant	Vector database
Claude Desktop	Testing prompts and workflows

Step 1: Run Qdrant

Since you already use Docker:

```

Bash
docker run -p 6333:6333 \
-v qdrant_storage:/qdrant/storage \
qdrant/qdrant
  
```

Qdrant API:

```

http://localhost:6333
  
```

Step 2: Run Ollama

Example models:

```

Bash
ollama pull llama3.1
ollama pull qwen3
ollama pull mistral
  
```

For enterprise knowledge assistants I currently prefer:

- Qwen 14B

- Llama 3.1 8B

If you have a powerful GPU:

```
Bash
ollama pull qwen3:14b
```

Step 3: Create Embeddings

Use a dedicated embedding model.

Example:

```
Bash
ollama pull nomic-embed-text
```

Workflow:

```
PDF
↓
Chunk Text
↓
Embedding
↓
Qdrant
```

Avoid using the chat model itself for embeddings.

Step 4: Document Ingestion Service

Build a small .NET API:

```
POST /upload
```

Process:

```
Upload PDF
↓
Extract text
↓
Chunk text
↓
Create embeddings
↓
Store vectors in Qdrant
↓
Store metadata in MSSQL
```

Metadata example:

```
SQL
Documents
-----
Id
Title
FileName
```

UploadedDate

Chunks

 ChunkId
 DocumentId
 PageNumber
 ChunkText
 VectorId

Step 5: Retrieval Pipeline

User asks:

How do I reset the billing workflow?

Backend:

```

Question
  ↓
Embedding
  ↓
Search Qdrant
  ↓
Top 5 Chunks
  ↓
Prompt Ollama
  ↓
Answer
  
```

Prompt example:

```

You are a company assistant.

Use ONLY the supplied context.

If the answer is unavailable,
say so.

Context:
{retrieved_chunks}

Question:
{user_question}
  
```

This reduces hallucinations significantly.

Step 6: Build the React Chat UI

Simple components:

ChatWindow
 MessageList
 MessageBubble
 InputBox
 SourcePanel

Answer format:

Answer

Sources:

- HR_Policy.pdf Page 8
- SOP_102.pdf Page 2

Users trust answers much more when citations are shown.

Step 7: Use LangFlow First

Before coding everything:

1. Connect Ollama.
2. Connect Qdrant.
3. Load sample documents.
4. Build a RAG flow.
5. Test retrieval quality.

Typical LangFlow flow:

```

Chat Input
  ↓
Retriever(Qdrant)
  ↓
Prompt Template
  ↓
Ollama
  ↓
Chat Output

```

Once the flow works, implement the same logic in code.

Step 8: Add Hybrid Search

Many teams miss this.

Use:

```

Vector Search
+
Keyword Search

```

Store keywords in MSSQL:

```

SQL
CONTAINS()
FREETEXT()

```

Then merge results.

Benefits:

- Finds exact IDs
- Finds part numbers

- Finds policy codes
- Better accuracy

Step 9: Add Permissions

If multiple departments exist:

```
HR
Finance
Engineering
Operations
```

Store permissions:

```
SQL
DocumentPermissions
-----
DocumentId
Role
```

Filter documents before retrieval.

Step 10: Production Docker Setup

```
YAML
services:
  frontend
  api
  qdrant
  ollama
  mssql
```

This gives a fully local AI assistant with:

- No cloud dependency
- Company data stays internal
- Fast retrieval
- Source citations
- Easy scaling

What I'd Build With Your Stack

Given your tools, my choice would be:

- Frontend: React + TypeScript
- Backend: ASP.NET Core 8 (Visual Studio)
- LLM: Ollama + Qwen 3 14B

- Embeddings: nomic-embed-text
- Vector DB: Qdrant
- Metadata/Auth: MSSQL
- Prototyping: LangFlow
- Deployment: Docker Compose

That architecture can comfortably support tens of thousands of documents and multiple users on a single server.

If you're building this for a company, I can also provide a complete folder structure, database schema, Docker Compose file, and ASP.NET Core API implementation plan.



yes

Great. Since you already have **Visual Studio + React + MSSQL + Ollama + Qdrant + Docker**, I'd recommend building it as a standard **3-tier RAG application** that can grow from a prototype into an enterprise knowledge assistant.

Project Structure

```
KnowledgeAssistant/
├── frontend/
│   ├── React TypeScript
│   ├── Chat UI
│   ├── Admin UI
│   └── Document Upload
├── backend/
│   ├── ASP.NET Core 8 Web API
│   ├── Authentication
│   ├── RAG Services
│   ├── Qdrant Service
│   ├── Ollama Service
│   └── MSSQL Repository
├── ingestion/
│   ├── PDF Processing
│   ├── Chunking
│   ├── Embedding Creation
│   └── Vector Upload
├── docker/
│   ├── docker-compose.yml
│   └── environment files
└── docs/
```

Database Design (MSSQL)

Users

SQL

```
CREATE TABLE Users
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    Email NVARCHAR(255),
    Name NVARCHAR(255),
    Role NVARCHAR(100)
);
```

Documents

SQL

```
CREATE TABLE Documents
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    Title NVARCHAR(500),
    FileName NVARCHAR(500),
    FilePath NVARCHAR(1000),
    UploadedBy UNIQUEIDENTIFIER,
    UploadDate DATETIME2,
    Status NVARCHAR(50)
);
```

Chunks

SQL

```
CREATE TABLE Chunks
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    DocumentId UNIQUEIDENTIFIER,
    ChunkIndex INT,
    PageNumber INT,
    ChunkText NVARCHAR(MAX),
    VectorId NVARCHAR(100)
);
```

Chat History

SQL

```
CREATE TABLE ChatMessages
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    UserId UNIQUEIDENTIFIER,
    ConversationId UNIQUEIDENTIFIER,
    MessageText NVARCHAR(MAX),
    MessageType NVARCHAR(50),
    CreatedDate DATETIME2
);
```

Docker Compose

YAML

```
version: '3.9'

services:
```

```
qdrant:
  image: qdrant/qdrant
  ports:
    - "6333:6333"
  volumes:
    - qdrant_data:/qdrant/storage

mssql:
  image: mcr.microsoft.com/mssql/server:2022-latest
  environment:
    SA_PASSWORD: YourStrongPassword123!
    ACCEPT_EULA: "Y"
  ports:
    - "1433:1433"

volumes:
  qdrant_data:
```

Ollama Models

Chat Model

```
Bash
ollama pull qwen3:14b
```

Good balance between quality and hardware requirements.

Embedding Model

```
Bash
ollama pull nomic-embed-text
```

ASP.NET Core Services

Folder Structure

```
Services/
├── OllamaService.cs
├── EmbeddingService.cs
├── QdrantService.cs
├── RetrievalService.cs
├── ChatService.cs
└── DocumentService.cs
```

Ollama Chat Service

```
C#
public interface IOllamaService
{
    Task<string> GenerateAsync(string prompt);
}
```

Endpoint:

```
http
POST http://localhost:11434/api/generate
```

Embedding Service

```
C#
public interface IEmbeddingService
{
    Task<float[]> CreateEmbeddingAsync(string text);
}
```

Endpoint:

```
http
POST /api/embeddings
```

Model:

```
nomiic-embed-text
```

Qdrant Collection

Create collection:

```
JSON
{
  "vectors": {
    "size": 768,
    "distance": "Cosine"
  }
}
```

Collection name:

```
knowledgebase
```

Document Upload Workflow

```
Upload PDF
  ↓
Extract Text
  ↓
Split Into Chunks
  ↓
Generate Embeddings
  ↓
Store in Qdrant
  ↓
Store Metadata in MSSQL
```

Chunking Strategy

Avoid huge chunks.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Recommended:

Chunk Size:
500-800 tokens

Overlap:
100 tokens

Example:

Chunk 1
0-800

Chunk 2
700-1500

Chunk 3
1400-2200

This preserves context between chunks.

PDF Extraction Libraries

For .NET:

- [PdfPig](#)
- [iText PDF Library](#)

PdfPig is a good starting point.

Retrieval Pipeline

When user asks:

How do I approve a leave request?

Process:

```

Question
↓
Embedding
↓
Qdrant Search
↓
Top 5 Chunks
↓
Prompt Assembly
↓
Qwen
↓
Response
  
```

Prompt Template

You are an internal company assistant.

Rules:

1. Answer only from provided context.
2. Do not invent information.
3. If answer not found, say:
"I couldn't find this information."

Context:

{chunks}

Question:

{question}

React Frontend

Pages:

/pages

ChatPage.tsx
AdminPage.tsx
UploadPage.tsx
LoginPage.tsx

Components:

/components

ChatWindow
MessageBubble
SourceList
UploadControl
Sidebar

Chat Response DTO

```
C#  
public class ChatResponse  
{  
    public string Answer { get; set; }  
  
    public List<SourceDto> Sources { get; set; }  
}
```

Source:

```
C#  
public class SourceDto  
{  
    public string DocumentName { get; set; }  
  
    public int PageNumber { get; set; }  
}
```

Frontend display:

Answer:
You can approve leave requests from HR Portal.

Sources:
Employee_Handbook.pdf (Page 14)
HR_Process.pdf (Page 3)

Advanced Features (Phase 2)

After the basic system works:

Hybrid Search

Combine:

- Qdrant vector search
- MSSQL full-text search

This improves retrieval of:

- Employee IDs
- Invoice numbers
- Product codes
- Policy references

Conversation Memory

Store:

ConversationId
UserId
Messages

Allow follow-up questions like:

What about managers?

without repeating context.

Role-Based Access

Example:

HR Documents
Finance Documents
Operations Documents

Filter retrieval before sending results to Qdrant.

Citations

Include chunk metadata:

JSON

```
{  
  "document": "Employee Handbook",  
  "page": 14  
}
```

Display citations under every answer.

Development Order

1. Setup MSSQL
2. Setup Qdrant
3. Setup Ollama
4. Build Upload API
5. Extract PDF text
6. Chunk documents
7. Generate embeddings
8. Store vectors
9. Build retrieval API
10. Build chat API
11. Build React UI
12. Add citations
13. Add authentication
14. Add permissions

This sequence gets you from zero to a working local AI knowledge assistant with source citations and document search in the shortest path.